

NITheCS Project

EVALUATING CONFORMAL PREDICTION FOR UNCERTAINTY QUANTIFICATION IN MACHINE LEARNING-BASED TEMPERATURE PREDICTION

Khethiwe Ntshangase

1. Import Libraries

```
In [39]: import pandas as pd
import numpy as np
import requests
import matplotlib.pyplot as plt
import seaborn as sns

from statsmodels.tsa.seasonal import STL

from sklearn.ensemble import RandomForestRegressor, VotingRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from xgboost import XGBRegressor

from mapie.regression import SplitConformalRegressor

import warnings
warnings.filterwarnings("ignore")
```

2. Load Dataset

```
In [40]: url = (
    "https://archive-api.open-meteo.com/v1/archive?"
    "latitude=-33.9833&longitude=18.6"
    "&start_date=2015-01-01"
    "&end_date=2025-12-31"
    "&daily=temperature_2m_max,temperature_2m_min,"
    "temperature_2m_mean,precipitation_sum"
    "&timezone=Africa/Johannesburg"
)

data = requests.get(url).json()

df = pd.DataFrame(data["daily"])
df["time"] = pd.to_datetime(df["time"])
```

```
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

print(df.head())
print(df.tail())
```

	time	temperature_2m_max	temperature_2m_min	temperature_2m_mean	precipitation_sum
0	2015-01-01	24.4	18.6	21.3	0.0
1	2015-01-02	23.5	17.3	20.4	0.0
2	2015-01-03	24.2	15.6	20.5	0.1
3	2015-01-04	23.5	17.7	20.3	1.1
4	2015-01-05	22.2	18.3	20.2	3.7

	time	temperature_2m_max	temperature_2m_min	temperature_2m_mean	precipitation_sum
4013	2025-12-27	27.9	17.0	22.4	0.0
4014	2025-12-28	30.5	16.8	23.7	0.0
4015	2025-12-29	35.6	17.9	27.1	0.0
4016	2025-12-30	30.2	19.1	22.9	0.0
4017	2025-12-31	26.8	18.5	22.0	0.0

```
In [41]: df = df.rename(columns={
    "temperature_2m_max": "TMAX",
    "temperature_2m_min": "TMIN",
    "temperature_2m_mean": "TMEAN",
    "precipitation_sum": "PRCP"})
df.head()
```

```
Out[41]:
```

	time	TMAX	TMIN	TMEAN	PRCP
0	2015-01-01	24.4	18.6	21.3	0.0
1	2015-01-02	23.5	17.3	20.4	0.0
2	2015-01-03	24.2	15.6	20.5	0.1
3	2015-01-04	23.5	17.7	20.3	1.1
4	2015-01-05	22.2	18.3	20.2	3.7

```
In [42]: df = df.sort_values("time")
df = df.reset_index(drop=True)
```

3. Exploratory Data Analysis (EDA)

```
In [43]: # Check for data information
print(df.info())

# Check data shape
print(df.shape)
```

```
<class 'pandas.DataFrame'>
RangeIndex: 4018 entries, 0 to 4017
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   time    4018 non-null    datetime64[us]
 1   TMAX    4018 non-null    float64
 2   TMIN    4018 non-null    float64
 3   TMEAN   4018 non-null    float64
 4   PRCP    4018 non-null    float64
dtypes: datetime64[us](1), float64(4)
memory usage: 157.1 KB
None
(4018, 5)
```

```
In [44]: # Check for duplicates
print(df.duplicated().sum())

# Check for missing values
print(df.isnull().sum())

# Check for unique values
print(df.nunique())

# Check for data types
print(df.dtypes)
```

```
0
time      0
TMAX      0
TMIN      0
TMEAN     0
PRCP      0
dtype: int64
time      4018
TMAX      250
TMIN      200
TMEAN     191
PRCP      204
dtype: int64
time      datetime64[us]
TMAX      float64
TMIN      float64
TMEAN     float64
PRCP      float64
dtype: object
```

Data Structure Interpretation

The dataset contains 4018 daily observations from 2015 to 2025.

No missing values or duplicate rows were found in the original dataset, indicating that the reanalysis data provided a complete and consistent time series for modelling.

In [45]: `print (df.describe())`

	time	TMAX	TMIN	TMEAN	PRCP
count	4018	4018.000000	4018.000000	4018.000000	4018.000000
mean	2020-07-01 12:00:00	22.227875	12.884818	17.204032	1.395495
min	2015-01-01 00:00:00	10.900000	0.000000	7.700000	0.000000
25%	2017-10-01 06:00:00	18.100000	10.100000	14.000000	0.000000
50%	2020-07-01 12:00:00	22.050000	13.000000	17.200000	0.000000
75%	2023-04-01 18:00:00	25.600000	15.900000	20.300000	0.500000
max	2025-12-31 00:00:00	37.400000	22.400000	28.000000	80.800000
std	NaN	5.022982	3.860272	3.893224	4.418035

Summary Statistics

The average daily maximum temperature was approximately 22.23°C, with values ranging from 10.9°C to 37.4°C.

Precipitation was generally low, with a median of 0 mm, indicating many dry days, although the maximum precipitation value of 80.8 mm shows that some days experienced heavy rainfall.

```
In [46]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

# Sort data
df = df.sort_values("time")

# Rolling average
df["TMAX_30day_avg"] = df["TMAX"].rolling(window=30).mean()

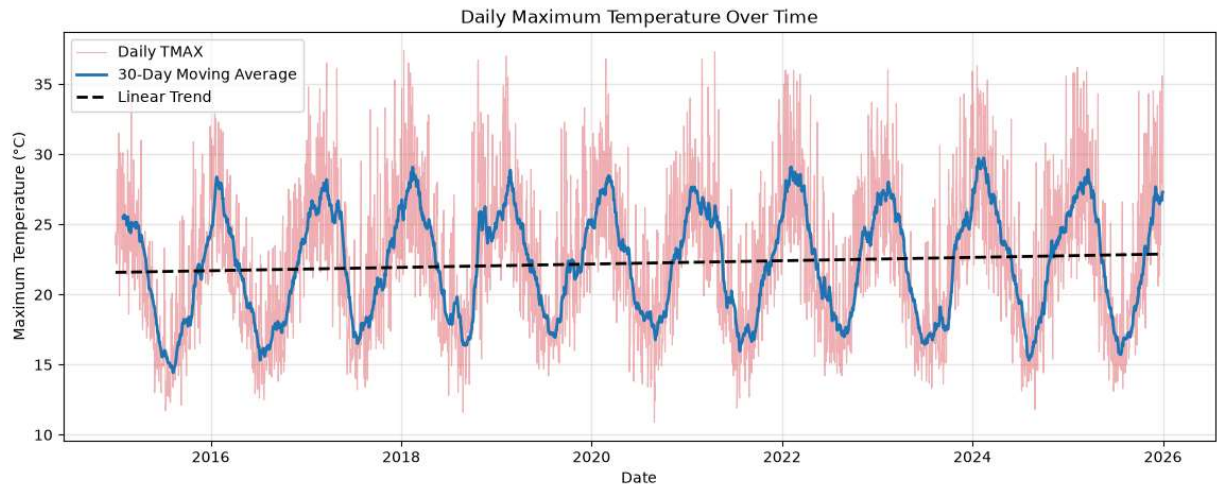
# Linear trend
x = mdates.date2num(df["time"])
y = df["TMAX"]

trend_coefficients = np.polyfit(x, y, 1)
trend_line = np.poly1d(trend_coefficients)

plt.figure(figsize=(14, 5))

plt.plot(df["time"], df["TMAX"], color="tab:red", linewidth=0.8, alpha=0.35, label="TMAX")
plt.plot(df["time"], df["TMAX_30day_avg"], color="tab:blue", linewidth=2, label="30-day avg")
plt.plot(df["time"], trend_line(x), color="black", linewidth=2, linestyle="--", label="Linear trend")

plt.title("Daily Maximum Temperature Over Time")
plt.xlabel("Date")
plt.ylabel("Maximum Temperature (°C)")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```



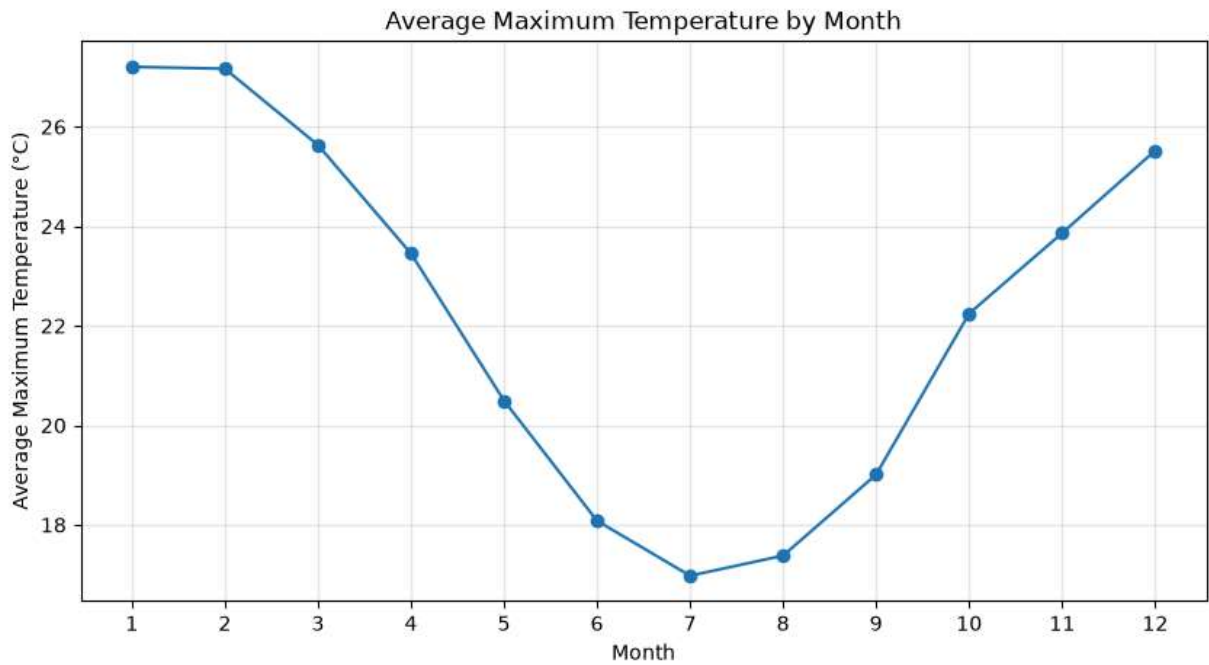
The daily maximum temperature series shows strong short-term fluctuations. The 30-day moving average smooths these daily changes and makes the seasonal pattern easier to observe. The repeated rises and falls over time indicate clear annual seasonality, with warmer and cooler periods recurring across years.

The linear trend provides a general indication of long-term direction, although the dominant pattern in the series appears to be seasonal variation rather than a strong linear trend.

```
In [47]: df["month"] = df["time"].dt.month

monthly_avg = df.groupby("month")["TMAX"].mean()

plt.figure(figsize=(10, 5))
plt.plot(monthly_avg.index, monthly_avg.values, marker="o", color="tab:blue")
plt.title("Average Maximum Temperature by Month")
plt.xlabel("Month")
plt.ylabel("Average Maximum Temperature (°C)")
plt.xticks(range(1, 13))
plt.grid(True, alpha=0.3)
plt.show()
```

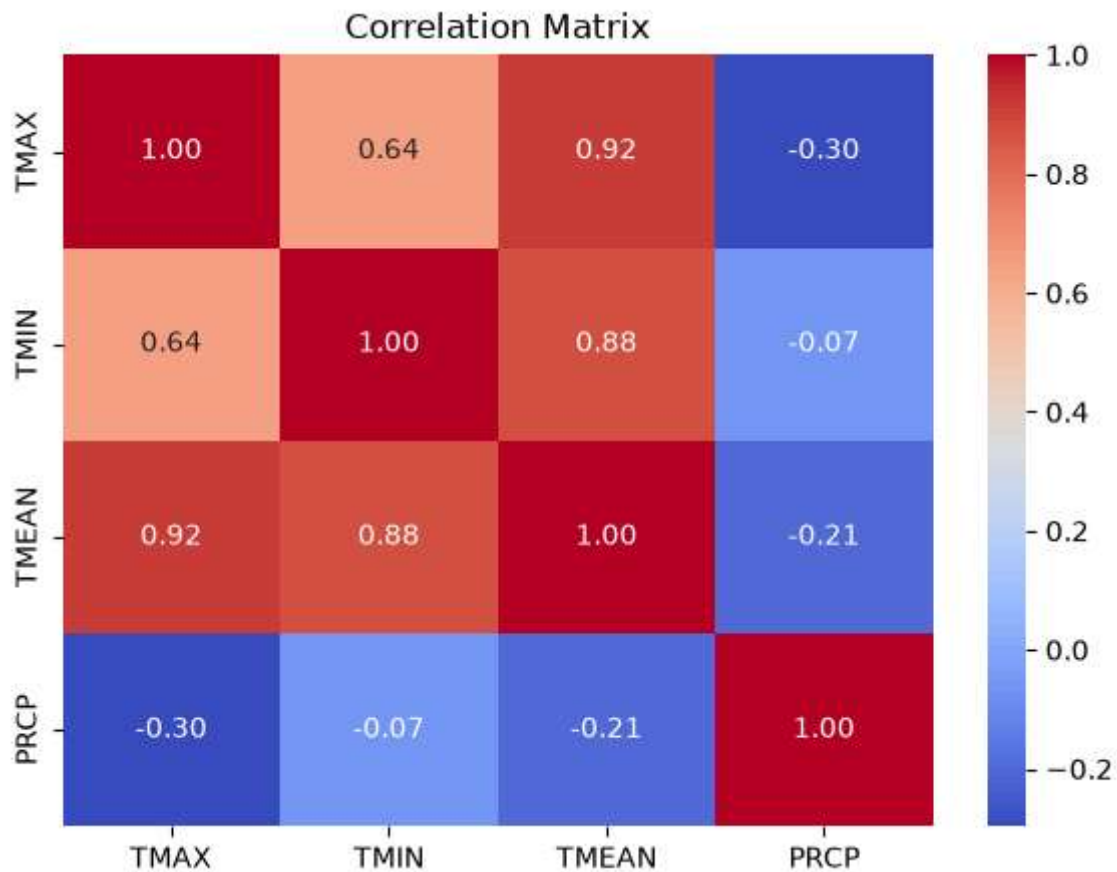


The monthly average plot shows variation in maximum temperature across months, confirming seasonal behaviour. Higher average temperatures occur during warmer months, while lower averages occur during cooler months. This supports the inclusion of seasonal time-based features in the models.

```
In [48]: corr = df[["TMAX", "TMIN", "TMEAN", "PRCP"]].corr()
print(corr)

plt.figure(figsize=(7, 5))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Matrix")
plt.show()
```

	TMAX	TMIN	TMEAN	PRCP
TMAX	1.000000	0.643191	0.916021	-0.297319
TMIN	0.643191	1.000000	0.877561	-0.065360
TMEAN	0.916021	0.877561	1.000000	-0.205795
PRCP	-0.297319	-0.065360	-0.205795	1.000000

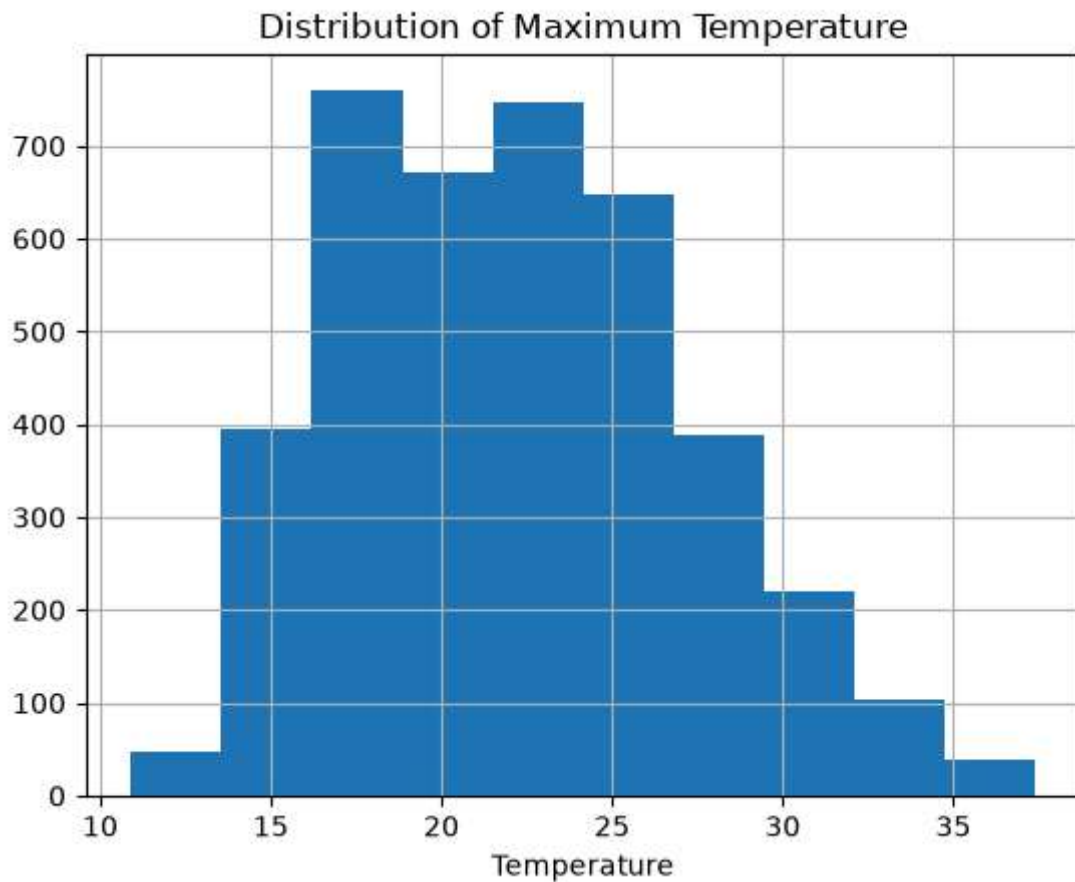


The monthly average plot shows variation in maximum temperature across months, confirming seasonal behaviour.

Higher average temperatures occur during warmer months, while lower averages occur during cooler months.

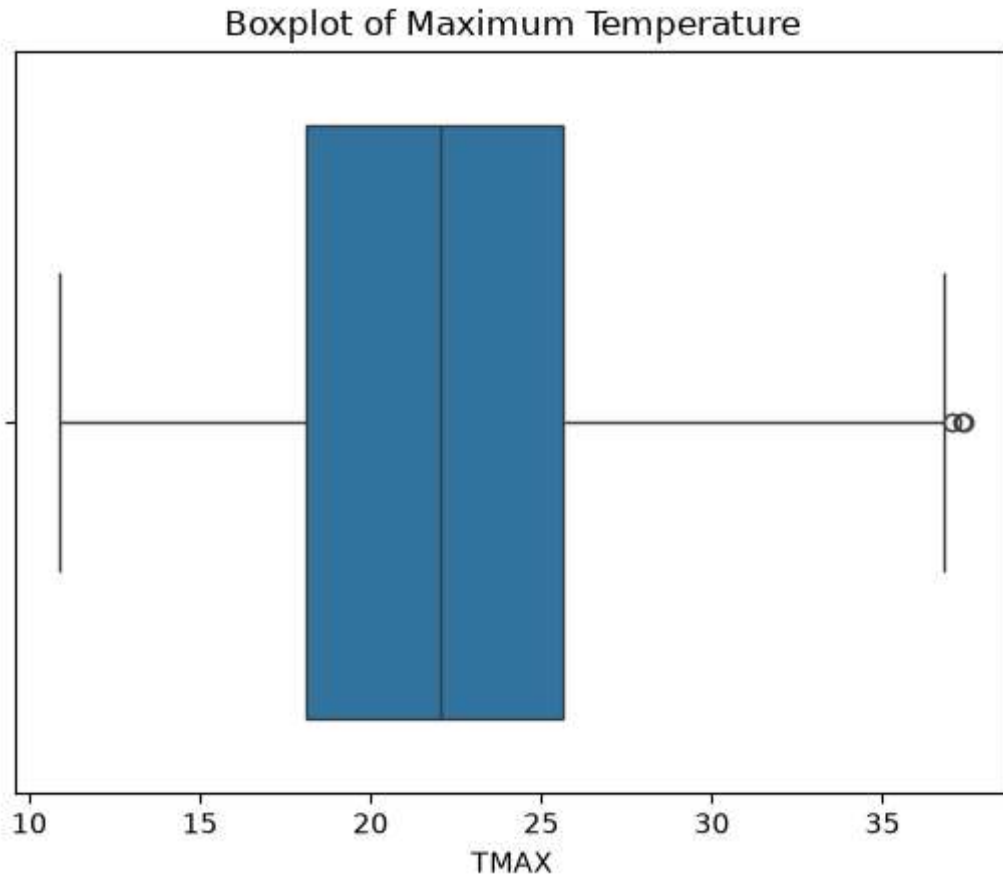
This supports the inclusion of seasonal time-based features in the models.

```
In [49]: df["TMAX"].hist()  
plt.title("Distribution of Maximum Temperature")  
plt.xlabel("Temperature")  
plt.show()
```



The distribution of daily maximum temperature is centred around approximately 22°C, with most values concentrated between 18°C and 26°C. The mean value of 22.23°C is very close to the median value of 22.05°C, suggesting that the distribution is reasonably balanced. However, the maximum value of 37.4°C indicates the presence of occasional hot days, which may contribute to a slight upper tail in the distribution.

```
In [50]: sns.boxplot(x=df["TMAX"])  
plt.title("Boxplot of Maximum Temperature")  
plt.show()
```

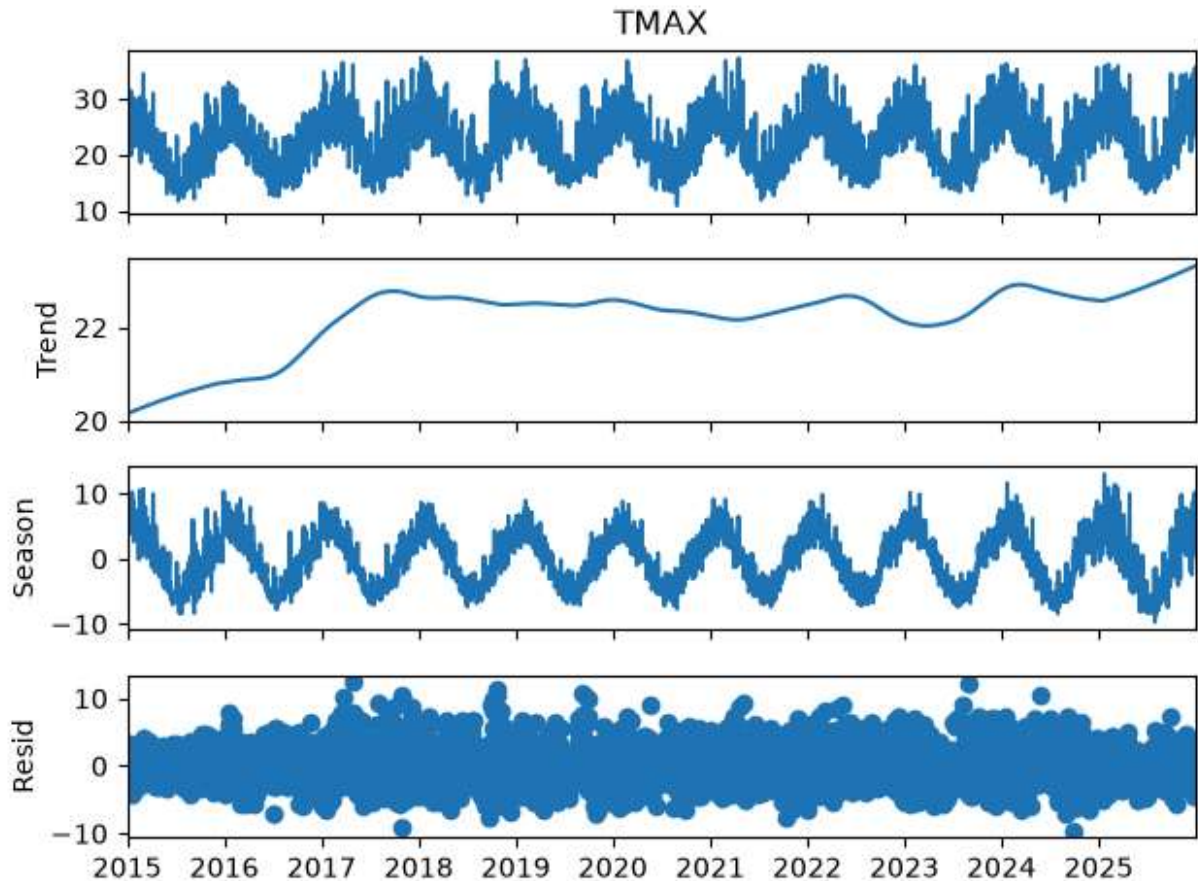


The boxplot shows that the median daily maximum temperature is approximately 22°C, with the middle 50% of values lying between 18.1°C and 25.6°C. This indicates moderate variability in daily maximum temperature. The upper end of the distribution extends to 37.4°C, which may appear as a high-temperature outlier according to the interquartile range rule. However, such values are physically plausible for hot days and were therefore retained in the analysis rather than treated as data errors.

```
In [51]: df_stl = df.set_index("time")

stl = STL(df_stl["TMAX"], period=365)
result = stl.fit()

result.plot()
plt.show()
```



STL decomposition was used to separate the TMAX time series into trend, seasonal, and residual components. The seasonal component helps confirm recurring annual temperature patterns, while the residual component captures irregular variation not explained by trend or seasonality.

Interpretation:

The observed component shows the original daily maximum temperature series. The series contains strong day-to-day fluctuations, but repeated annual rises and falls are visible, indicating seasonal variation in maximum temperature.

Trend component: The trend component captures the longer-term movement in maximum temperature after the seasonal pattern has been removed. The trend appears smoother than the observed series and suggests gradual variation over time rather than abrupt changes. This indicates that while long-term variation exists, the dominant pattern in the data is seasonal.

Seasonal component: The seasonal component shows a clear repeating annual pattern, confirming that maximum temperature follows a strong seasonal cycle. This supports the inclusion of cyclical time-based features, such as sine and cosine transformations of month and day of year, in the machine learning models

Residual component: The residual component represents irregular variation that is not explained by the trend or seasonal components. The presence of residual fluctuations suggests that daily maximum temperature contains short-term variability, which supports the use of machine learning models capable of capturing nonlinear and complex relationships.

4. Feature Engineering

```
In [52]: df["TMAX_next"] = df["TMAX"].shift(-1)
```

```
In [53]: df["TMAX_lag1"] = df["TMAX"].shift(1)
df["TMAX_lag2"] = df["TMAX"].shift(2)
df["TMAX_lag3"] = df["TMAX"].shift(3)
```

```
In [54]: df["month"] = df["time"].dt.month
df["day"] = df["time"].dt.dayofyear
```

```
In [55]: df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)

df["day_sin"] = np.sin(2 * np.pi * df["day"] / 365)
df["day_cos"] = np.cos(2 * np.pi * df["day"] / 365)
```

```
In [56]: df.isnull().sum()
```

```
Out[56]: time          0
TMAX              0
TMIN              0
TMEAN            0
PRCP             0
TMAX_30day_avg    29
month            0
TMAX_next        1
TMAX_lag1        1
TMAX_lag2        2
TMAX_lag3        3
day              0
month_sin        0
month_cos        0
day_sin          0
day_cos          0
dtype: int64
```

```
In [57]: model_columns = [
    "TMAX_next",
    "TMAX_lag1",
    "TMAX_lag2",
    "TMAX_lag3",
    "TMIN",
    "TMEAN",
    "PRCP",
    "month",
```

```

    "day",
    "month_sin",
    "month_cos",
    "day_sin",
    "day_cos"
]

df_model = df.dropna(subset=model_columns).copy()
df_model = df_model.sort_values("time").reset_index(drop=True)

```

5. Define Features and Target

```

In [58]: features = ["TMIN", "TMEAN", "PRCP", "TMAX_lag1", "TMAX_lag2", "TMAX_lag3", "month_
X = df_model[features]
y = df_model["TMAX_next"]

```

```

In [59]: dates = df_model["time"]

```

6. Split Data

```

In [60]: print("Length of df:", len(df))
print("Length of df_model:", len(df_model))

```

Length of df: 4018
Length of df_model: 4014

```

In [61]: n = len(df_model)

train_end = int(n * 0.70)
cal_end = int(n * 0.85)

X_train = X.iloc[:train_end]
y_train = y.iloc[:train_end]

X_cal = X.iloc[train_end:cal_end]
y_cal = y.iloc[train_end:cal_end]

X_test = X.iloc[cal_end:]
y_test = y.iloc[cal_end:]

dates_test = dates.iloc[cal_end:]

print("Training size:", len(X_train))
print("Calibration size:", len(X_cal))
print("Test size:", len(X_test))

```

Training size: 2809
Calibration size: 602
Test size: 603

```

In [62]: print("Training dates:")
print(dates.iloc[:train_end].min(), "to", dates.iloc[:train_end].max())

print("Calibration dates:")

```

```
print(dates.iloc[train_end:cal_end].min(), "to", dates.iloc[train_end:cal_end].max())

print("Test dates:")
print(dates.iloc[cal_end:].min(), "to", dates.iloc[cal_end:].max())
```

Training dates:

2015-01-04 00:00:00 to 2022-09-12 00:00:00

Calibration dates:

2022-09-13 00:00:00 to 2024-05-06 00:00:00

Test dates:

2024-05-07 00:00:00 to 2025-12-30 00:00:00

The data was split chronologically, with the earliest observations used for training, the middle period used for calibration, and the most recent period used for testing. This preserved the temporal order of the data and avoided data leakage.

7. Define models

```
In [ ]: models = {
    "Random Forest": RandomForestRegressor(
        n_estimators=300,
        random_state=42),

    "XGBoost": XGBRegressor(
        n_estimators=300,
        learning_rate=0.05,
        max_depth=5,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42),

    "MLP": Pipeline([
        ("scaler", StandardScaler()),
        ("mlp", MLPRegressor(
            hidden_layer_sizes=(64, 32),
            activation="relu",
            max_iter=1000,
            random_state=42))])}]
```

8. Fit all models and evaluate point predictions

```
In [64]: point_results = []
trained_models = {}

for name, model in models.items():
    print(f"Training {name}...")

    model.fit(X_train, y_train)
    trained_models[name] = model

    y_pred = model.predict(X_test)

    mae = mean_absolute_error(y_test, y_pred)
```

```

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

point_results.append({
    "Model": name,
    "MAE": mae,
    "RMSE": rmse,
    "R2": r2
})

point_results_df = pd.DataFrame(point_results)
point_results_df

```

Training Random Forest...

Training XGBoost...

Training MLP...

Out[64]:

	Model	MAE	RMSE	R2
0	Random Forest	2.363144	3.096867	0.654461
1	XGBoost	2.361364	3.122658	0.648682
2	MLP	2.272672	3.073585	0.659637

8. Apply Conformal Prediction to all three models

```

In [ ]: conformal_results = {}
summary_results = []

confidence_level = 0.90

for name, model in trained_models.items():
    print(f"Applying conformal prediction to {name}...")

    cp_model = SplitConformalRegressor(
        estimator=model,
        confidence_level=confidence_level,
        prefit=True)

    cp_model.conformalize(X_cal, y_cal)

    y_pred, intervals = cp_model.predict_interval(X_test)

    lower = intervals[:, 0, 0]
    upper = intervals[:, 1, 0]

    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)

    coverage = ((y_test >= lower) & (y_test <= upper)).mean()
    avg_width = np.mean(upper - lower)

    summary_results.append({
        "Model": name,

```

```

    "MAE": mae,
    "RMSE": rmse,
    "R2": r2,
    "Coverage": coverage,
    "Average Interval Width": avg_width})

conformal_results[name] = {
    "y_pred": y_pred,
    "lower": lower,
    "upper": upper}

results_df = pd.DataFrame(summary_results)
results_df

```

Applying conformal prediction to Random Forest...

Applying conformal prediction to XGBoost...

Applying conformal prediction to MLP...

Out[]:

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
0	Random Forest	2.363144	3.096867	0.654461	0.892206	10.184667
1	XGBoost	2.361364	3.122658	0.648682	0.903814	10.585945
2	MLP	2.272672	3.073585	0.659637	0.902156	10.289621

9. Model comparison tables

In [66]: `results_df.sort_values(by="MAE")`

Out[66]:

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
2	MLP	2.272672	3.073585	0.659637	0.902156	10.289621
1	XGBoost	2.361364	3.122658	0.648682	0.903814	10.585945
0	Random Forest	2.363144	3.096867	0.654461	0.892206	10.184667

In [67]: `results_df.sort_values(by="RMSE")`

Out[67]:

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
2	MLP	2.272672	3.073585	0.659637	0.902156	10.289621
0	Random Forest	2.363144	3.096867	0.654461	0.892206	10.184667
1	XGBoost	2.361364	3.122658	0.648682	0.903814	10.585945

In [68]: `results_df.sort_values(by="Average Interval Width")`

Out[68]:

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
0	Random Forest	2.363144	3.096867	0.654461	0.892206	10.184667
2	MLP	2.272672	3.073585	0.659637	0.902156	10.289621
1	XGBoost	2.361364	3.122658	0.648682	0.903814	10.585945

10. Plot predictions and conformal intervals on (XGBoost)

```
In [ ]: model_name = "MLP"

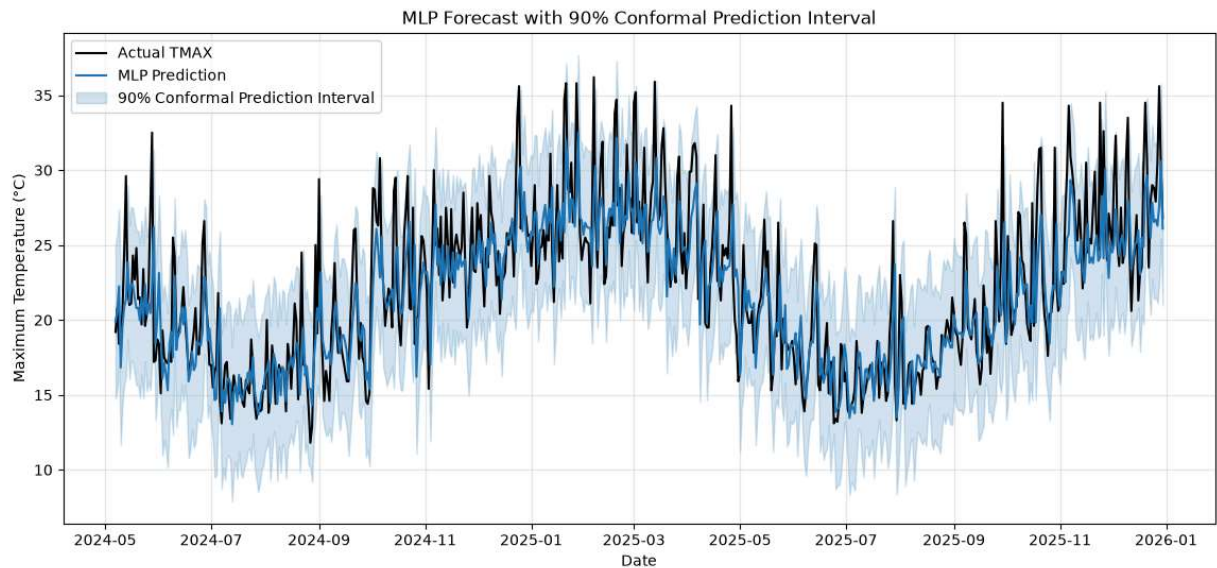
pred = conformal_results[model_name]["y_pred"]
lower = conformal_results[model_name]["lower"]
upper = conformal_results[model_name]["upper"]

plt.figure(figsize=(14, 6))

plt.plot(dates_test, y_test.values, label="Actual TMAX", color="black", linewidth=1)
plt.plot(dates_test, pred, label=f"{model_name} Prediction", color="tab:blue", line

plt.fill_between(
    dates_test,
    lower,
    upper,
    color="tab:blue",
    alpha=0.2,
    label="90% Conformal Prediction Interval")

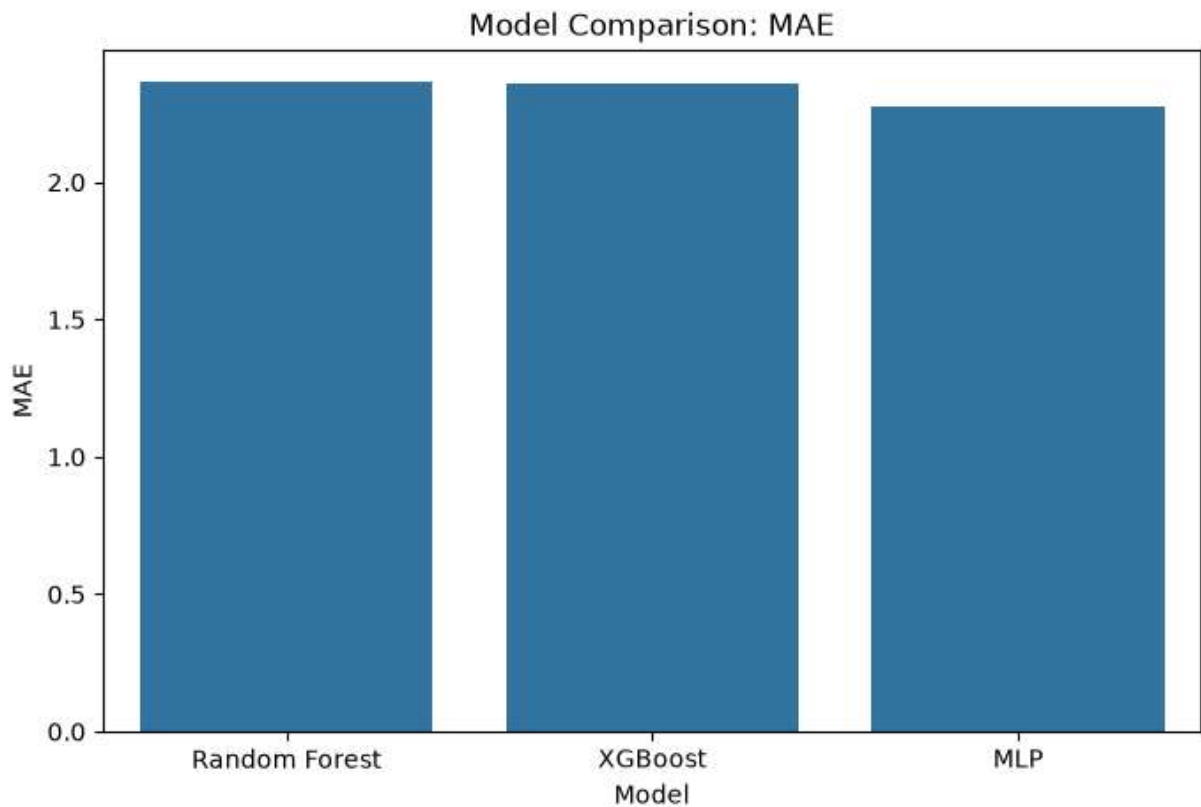
plt.title(f"{model_name} Forecast with 90% Conformal Prediction Interval")
plt.xlabel("Date")
plt.ylabel("Maximum Temperature (°C)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



11. Plot all model results as bar charts

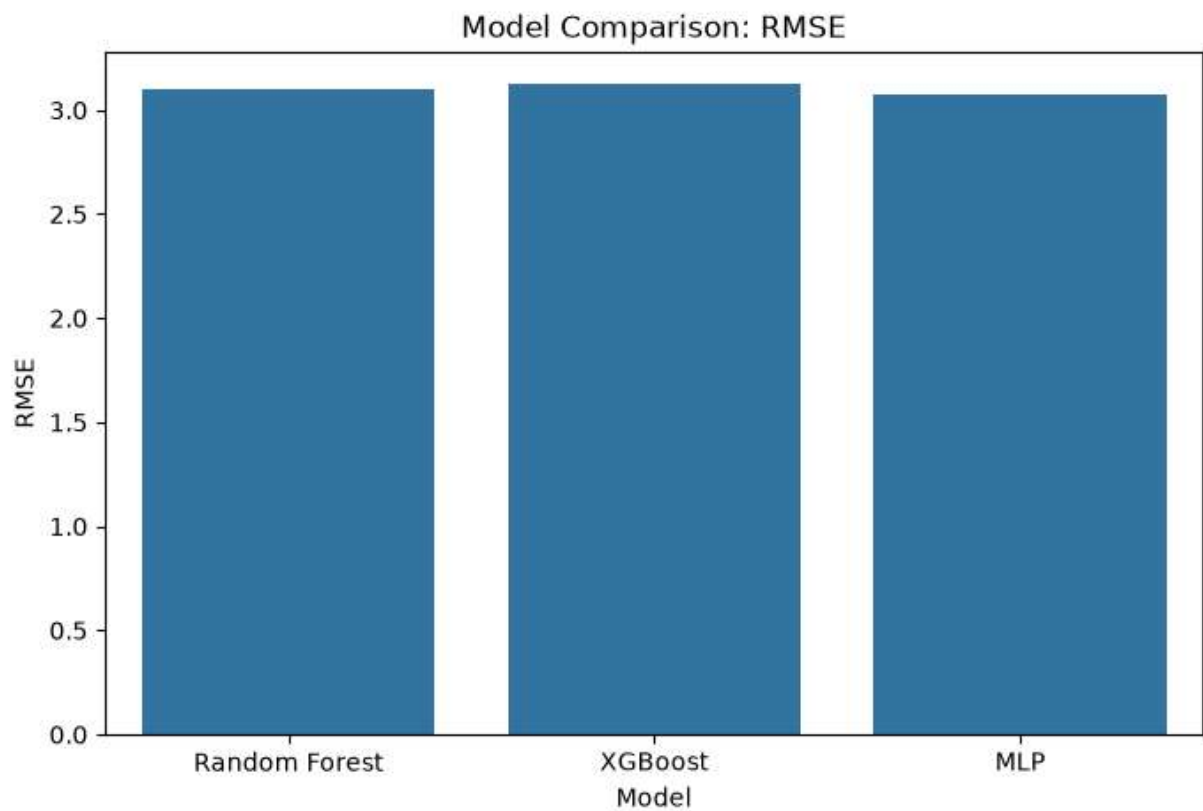
MAE comparison

```
In [70]: plt.figure(figsize=(8, 5))
sns.barplot(data=results_df, x="Model", y="MAE")
plt.title("Model Comparison: MAE")
plt.ylabel("MAE")
plt.show()
```



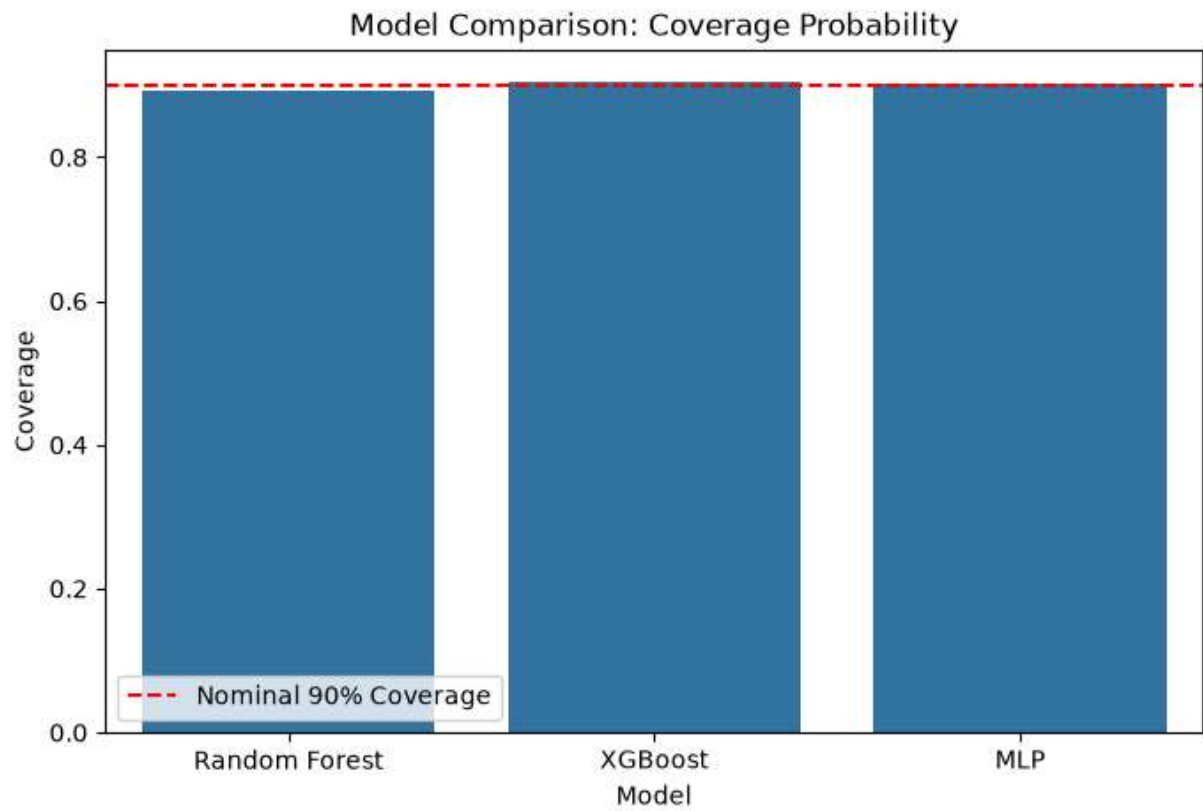
RMSE comparison

```
In [71]: plt.figure(figsize=(8, 5))
sns.barplot(data=results_df, x="Model", y="RMSE")
plt.title("Model Comparison: RMSE")
plt.ylabel("RMSE")
plt.show()
```



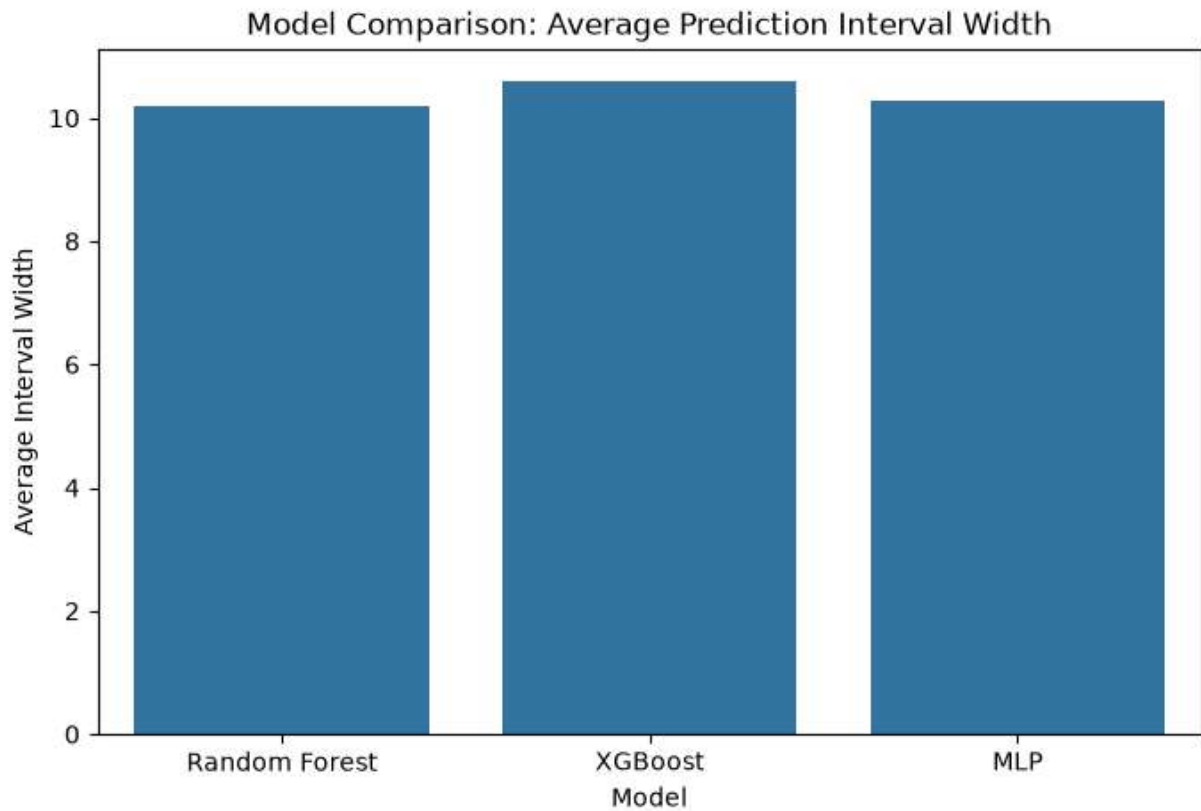
Coverage comparison

```
In [72]: plt.figure(figsize=(8, 5))
sns.barplot(data=results_df, x="Model", y="Coverage")
plt.axhline(0.90, color="red", linestyle="--", label="Nominal 90% Coverage")
plt.title("Model Comparison: Coverage Probability")
plt.ylabel("Coverage")
plt.legend()
plt.show()
```



Interval width comparison

```
In [73]: plt.figure(figsize=(8, 5))
sns.barplot(data=results_df, x="Model", y="Average Interval Width")
plt.title("Model Comparison: Average Prediction Interval Width")
plt.ylabel("Average Interval Width")
plt.show()
```



Before hyperparameter tuning, the MLP model achieved the best predictive performance. It produced the lowest MAE of 2.273°C, the lowest RMSE of 3.074°C, and the highest R^2 value of 0.660. This indicates that, among the three initial models, MLP made the smallest average prediction errors and explained the largest proportion of variation in next-day maximum temperature.

Random Forest and XGBoost performed similarly in terms of MAE, with values of 2.363°C and 2.361°C, respectively. However, Random Forest had a slightly better RMSE and R^2 than XGBoost. This suggests that although XGBoost had marginally lower average absolute error, Random Forest handled larger errors slightly better in the untuned setting.

For conformal prediction performance, all three models produced coverage close to the target 90% level. XGBoost achieved the highest coverage at 90.38%, meaning its prediction intervals captured slightly more than the required proportion of actual observations. However, this came with the widest average interval width of 10.586°C. This suggests that XGBoost was more conservative: it achieved higher coverage partly by producing wider intervals. MLP achieved coverage of 90.22% with a slightly narrower interval width than XGBoost. This means that before tuning, MLP offered the best balance between predictive accuracy and uncertainty reliability.

12. Hyperparameter Tuning

```
In [ ]: from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
        from sklearn.ensemble import RandomForestRegressor
```

```

from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from xgboost import XGBRegressor

tscv = TimeSeriesSplit(n_splits=5)

models = {
    "Random Forest": {
        "model": RandomForestRegressor(random_state=42),
        "params": {
            "n_estimators": [100, 300],
            "max_depth": [None, 10, 20],
            "min_samples_leaf": [1, 2]}},

    "XGBoost": {
        "model": XGBRegressor(
            objective="reg:squarederror",
            random_state=42),
        "params": {
            "n_estimators": [100, 300],
            "learning_rate": [0.05, 0.1],
            "max_depth": [3, 5]}},

    "MLP": {
        "model": Pipeline([
            ("scaler", StandardScaler()),
            ("mlp", MLPRegressor(
                max_iter=1000,
                random_state=42
            ))]),
        "params": {
            "mlp__hidden_layer_sizes": [(64,), (64, 32)],
            "mlp__activation": ["relu"],
            "mlp__learning_rate_init": [0.001],
            "mlp__alpha": [0.0001, 0.001]}}}

```

```

In [ ]: best_models = {}
tuning_results = []

for name, item in models.items():
    print(f"Tuning {name}...")

    grid = GridSearchCV(
        estimator=item["model"],
        param_grid=item["params"],
        cv=tscv,
        scoring="neg_mean_absolute_error",
        n_jobs=-1)

    grid.fit(X_train, y_train)

    best_models[name] = grid.best_estimator_

    tuning_results.append({
        "Model": name,

```

```

    "Best Parameters": grid.best_params_,
    "Best CV MAE": -grid.best_score_})

tuning_results_df = pd.DataFrame(tuning_results)
tuning_results_df

```

Tuning Random Forest...

Tuning XGBoost...

Tuning MLP...

```

Out[ ]:

```

	Model	Best Parameters	Best CV MAE
0	Random Forest	{'max_depth': 10, 'min_samples_leaf': 2, 'n_es...	2.319575
1	XGBoost	{'learning_rate': 0.05, 'max_depth': 3, 'n_est...	2.299902
2	MLP	{'mlp_activation': 'relu', 'mlp_alpha': 0.00...	2.362260

The tuning process used MAE as the optimisation criterion because MAE is directly interpretable as average temperature error in degrees Celsius. During cross-validation on the training set, XGBoost achieved the lowest CV MAE of 2.300°C, followed by Random Forest with 2.320°C, and MLP with 2.362°C.

This suggests that, during the internal validation stage, XGBoost had the strongest average validation performance. However, cross-validation results are not the final performance results. The final conclusion must be based on how the tuned models perform on the independent test set, especially after conformal prediction is applied.

A key point here is that the model that performs best during cross-validation does not always perform best on the final test period. This can happen because the test period represents a later time block and may contain different seasonal or weather behaviour from the training folds.

Apply Conformal prediction to tuned models

```

In [ ]: results = []
tuned_conformal_results = {}

confidence_level = 0.90

for name, model in best_models.items():
    print(f"Applying conformal prediction to {name}...")

    cp_model = SplitConformalRegressor(
        estimator=model,
        confidence_level=confidence_level,
        prefit=True)

    cp_model.conformalize(X_cal, y_cal)

    y_pred, intervals = cp_model.predict_interval(X_test)

```

```

lower = intervals[:, 0, 0]
upper = intervals[:, 1, 0]

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

coverage = ((y_test >= lower) & (y_test <= upper)).mean()
avg_width = np.mean(upper - lower)

results.append({
    "Model": name,
    "MAE": mae,
    "RMSE": rmse,
    "R2": r2,
    "Coverage": coverage,
    "Average Interval Width": avg_width})

tuned_conformal_results[name] = {
    "Prediction": y_pred,
    "Lower": lower,
    "Upper": upper}

tuned_results_df = pd.DataFrame(results)
tuned_results_df

```

Applying conformal prediction to Random Forest...

Applying conformal prediction to XGBoost...

Applying conformal prediction to MLP...

Out[]:

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
0	Random Forest	2.327295	3.068703	0.660717	0.885572	9.923735
1	XGBoost	2.313274	3.089012	0.656212	0.883914	9.681747
2	MLP	2.267956	3.038078	0.667455	0.888889	9.611099

After hyperparameter tuning, all three models improved in terms of interval width, and the predictive performance improved slightly compared with the untuned versions. The tuned MLP model remained the strongest overall performer, with the lowest MAE of 2.268°C, the lowest RMSE of 3.038°C, and the highest R^2 of 0.667.

This means that the tuned MLP model had the smallest average prediction error and explained approximately 66.7% of the variation in next-day maximum temperature. In practical terms, the model's predictions were, on average, about 2.27°C away from the observed next-day maximum temperature. The tuned XGBoost model also improved compared with its untuned version, reducing MAE from 2.361°C to 2.313°C and reducing the average interval width from 10.586°C to 9.682°C. However, its RMSE and R^2 remained weaker than MLP. This suggests that XGBoost improved after tuning but still did not outperform MLP on the final test set.

Random Forest also improved after tuning, with MAE decreasing from 2.363°C to 2.327°C, RMSE decreasing from 3.097°C to 3.069°C, and R^2 increasing from 0.654 to 0.661. This shows that tuning helped Random Forest, although it still remained behind MLP in overall predictive accuracy.

Best Performing tuned model

```
In [77]: tuned_results_df.sort_values(by="MAE")
```

```
Out[77]:
```

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
2	MLP	2.267956	3.038078	0.667455	0.888889	9.611099
1	XGBoost	2.313274	3.089012	0.656212	0.883914	9.681747
0	Random Forest	2.327295	3.068703	0.660717	0.885572	9.923735

```
In [78]: tuned_results_df.sort_values(by="RMSE")
```

```
Out[78]:
```

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
2	MLP	2.267956	3.038078	0.667455	0.888889	9.611099
0	Random Forest	2.327295	3.068703	0.660717	0.885572	9.923735
1	XGBoost	2.313274	3.089012	0.656212	0.883914	9.681747

```
In [79]: tuned_results_df.sort_values(by="Average Interval Width")
```

```
Out[79]:
```

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
2	MLP	2.267956	3.038078	0.667455	0.888889	9.611099
1	XGBoost	2.313274	3.089012	0.656212	0.883914	9.681747
0	Random Forest	2.327295	3.068703	0.660717	0.885572	9.923735

After tuning, the prediction intervals became narrower, but the empirical coverage also decreased slightly below the target 90% level. This shows a common trade-off in uncertainty quantification: narrower intervals are more efficient, but they may cover slightly fewer true values. The tuned MLP model produced the narrowest interval width of 9.611°C, while maintaining coverage of 88.89%. This is slightly below the nominal 90% target but still very close. The result suggests that MLP produced the sharpest intervals while still providing reasonably reliable uncertainty estimates.

XGBoost had the second narrowest intervals after tuning, with an average width of 9.682°C, but its coverage dropped to 88.39%. This means that although XGBoost became more efficient in terms of interval width, it undercovered slightly more than MLP. Random Forest had the widest tuned intervals among the three models, with an average width of 9.924°C,

and coverage of 88.56%. Overall, tuning improved interval efficiency by narrowing the intervals, but this came at the cost of slightly reduced coverage.

```
In [ ]: tuned_conformal_results = {}
tuned_summary_results = []

confidence_level = 0.9

for name, model in best_models.items():
    print(f"Applying conformal prediction to tuned {name}...")

    cp_model = SplitConformalRegressor(
        estimator=model,
        confidence_level=confidence_level,
        prefit=True)

    cp_model.conformalize(X_cal, y_cal)

    y_pred, intervals = cp_model.predict_interval(X_test)

    lower = intervals[:, 0, 0]
    upper = intervals[:, 1, 0]

    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)

    coverage = ((y_test >= lower) & (y_test <= upper)).mean()
    avg_width = np.mean(upper - lower)

    tuned_summary_results.append({
        "Model": name,
        "MAE": mae,
        "RMSE": rmse,
        "R2": r2,
        "Coverage": coverage,
        "Average Interval Width": avg_width})

    tuned_conformal_results[name] = {
        "y_pred": y_pred,
        "lower": lower,
        "upper": upper}

tuned_results_df = pd.DataFrame(tuned_summary_results)
tuned_results_df
```

Applying conformal prediction to tuned Random Forest...
 Applying conformal prediction to tuned XGBoost...
 Applying conformal prediction to tuned MLP...

Out []:

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
0	Random Forest	2.327295	3.068703	0.660717	0.885572	9.923735
1	XGBoost	2.313274	3.089012	0.656212	0.883914	9.681747
2	MLP	2.267956	3.038078	0.667455	0.888889	9.611099

```
In [ ]: model_name = "MLP"

pred = tuned_conformal_results[model_name]["y_pred"]
lower = tuned_conformal_results[model_name]["lower"]
upper = tuned_conformal_results[model_name]["upper"]

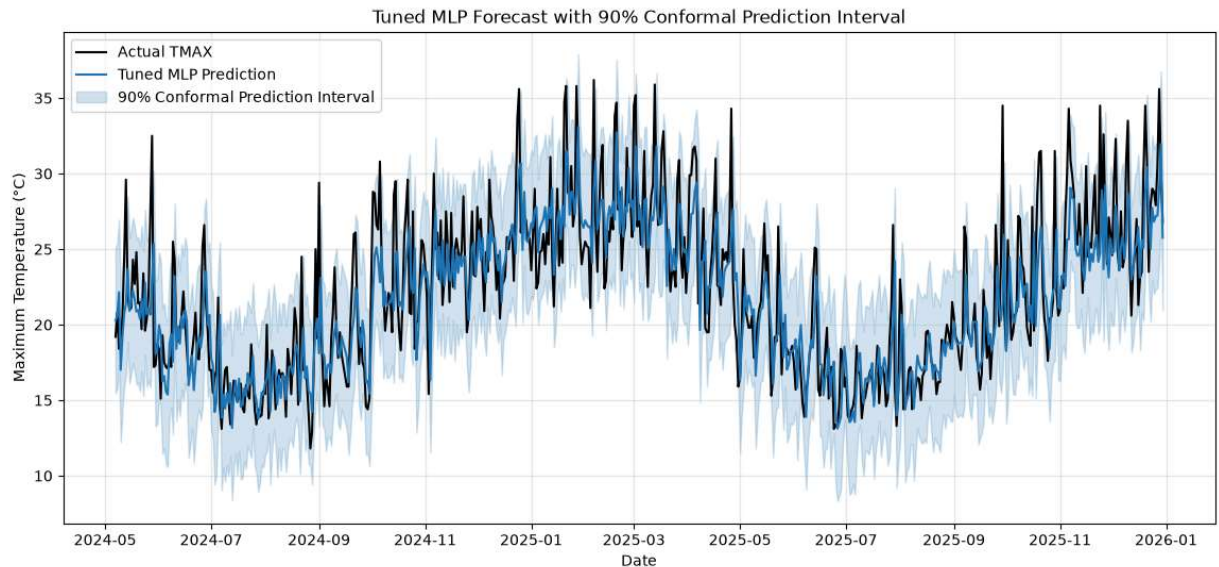
plt.figure(figsize=(14, 6))

plt.plot(
    dates_test,
    y_test.values,
    label="Actual TMAX",
    color="black",
    linewidth=1.5)

plt.plot(
    dates_test,
    pred,
    label="Tuned MLP Prediction",
    color="tab:blue",
    linewidth=1.5)

plt.fill_between(
    dates_test,
    lower,
    upper,
    color="tab:blue",
    alpha=0.2,
    label="90% Conformal Prediction Interval")

plt.title("Tuned MLP Forecast with 90% Conformal Prediction Interval")
plt.xlabel("Date")
plt.ylabel("Maximum Temperature (°C)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



```
In [ ]: model_name = "MLP"
n_plot = 120

pred = tuned_conformal_results[model_name]["y_pred"]
lower = tuned_conformal_results[model_name]["lower"]
upper = tuned_conformal_results[model_name]["upper"]

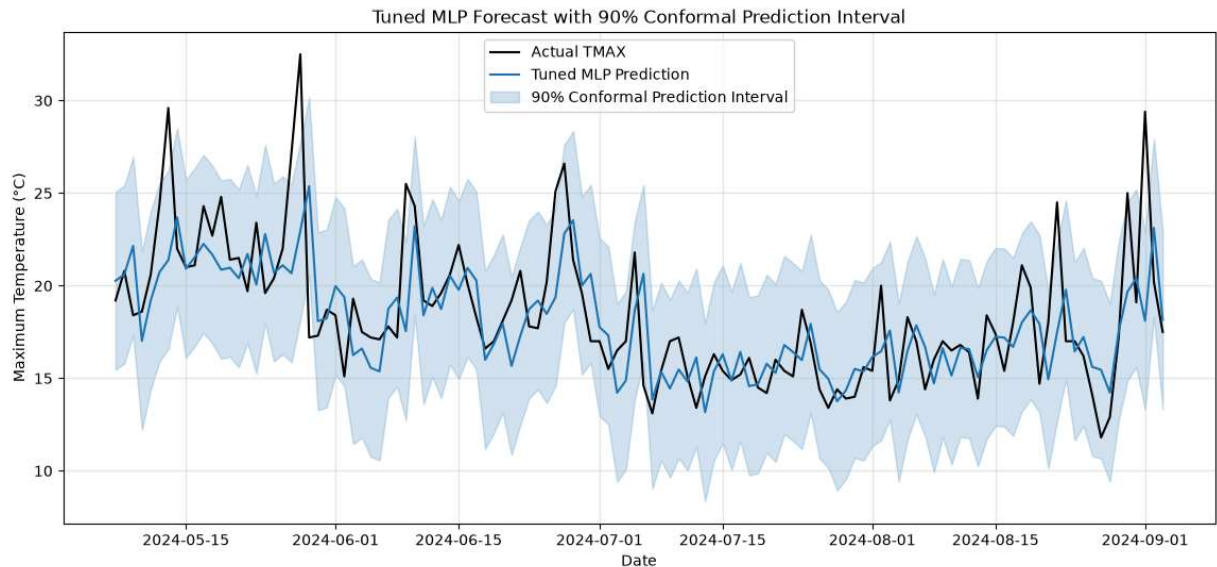
plt.figure(figsize=(14, 6))

plt.plot(
    dates_test.iloc[:n_plot],
    y_test.values[:n_plot],
    label="Actual TMAX",
    color="black",
    linewidth=1.5)

plt.plot(
    dates_test.iloc[:n_plot],
    pred[:n_plot],
    label="Tuned MLP Prediction",
    color="tab:blue",
    linewidth=1.5)

plt.fill_between(
    dates_test.iloc[:n_plot],
    lower[:n_plot],
    upper[:n_plot],
    color="tab:blue",
    alpha=0.2,
    label="90% Conformal Prediction Interval")

plt.title("Tuned MLP Forecast with 90% Conformal Prediction Interval")
plt.xlabel("Date")
plt.ylabel("Maximum Temperature (°C)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



```
In [83]: print(best_models.keys())

dict_keys(['Random Forest', 'XGBoost', 'MLP'])
```

Save Results

```
In [ ]: df_model.to_csv("processed_temperature_data.csv", index=False)

In [ ]: tuning_results_df.to_csv("hyperparameter_tuning_results.csv", index=False)
tuned_results_df.to_csv("tuned_conformal_results.csv", index=False)

In [ ]: # Persistence baseline: predict tomorrow's TMAX using today's TMAX
baseline_pred = df_model["TMAX"].iloc[cal_end:].values

baseline_mae = mean_absolute_error(y_test, baseline_pred)
baseline_rmse = np.sqrt(mean_squared_error(y_test, baseline_pred))
baseline_r2 = r2_score(y_test, baseline_pred)

print("Persistence Baseline")
print("MAE:", baseline_mae)
print("RMSE:", baseline_rmse)
print("R2:", baseline_r2)
```

```
Persistence Baseline
MAE: 2.7112769485903816
RMSE: 3.676912415164303
R2: 0.5128994994388167
```

Best Performing Model

Overall, the tuned MLP model provided the strongest performance among the models evaluated. It achieved the lowest MAE of 2.268°C, the lowest RMSE of 3.038°C, and the highest R^2 value of 0.667, indicating that it produced the most accurate point forecasts for next-day maximum temperature. It also produced the narrowest average conformal prediction intervals, with an average width of 9.611°C. Although its empirical coverage of

88.89% was slightly below the nominal 90% level, it remained close to the target and represented the best balance between predictive accuracy and interval efficiency. XGBoost achieved strong validation performance during hyperparameter tuning, but it did not outperform MLP on the final test set. Random Forest showed moderate performance and improved after tuning, but remained weaker than MLP overall.

Conformal Prediction Interpretation

Split conformal prediction produced prediction intervals with empirical coverage close to the nominal 90% level. Before hyperparameter tuning, XGBoost and MLP slightly exceeded the 90% target, achieving coverage values of 90.38% and 90.22%, respectively. After tuning, interval widths became narrower across all models, but empirical coverage decreased slightly below 90%. This indicates a trade-off between interval efficiency and coverage. The tuned MLP model achieved the narrowest intervals while maintaining coverage close to the target level, suggesting that it provided the most efficient uncertainty estimates among the tuned models. The slight undercoverage may be explained by the time-series nature of the data, where strict exchangeability is difficult to guarantee.

Effect of Confidence Level on Prediction Interval Width and Coverage

```
In [ ]: confidence_levels = [0.80, 0.90, 0.95]

coverage_results = []

for confidence_level in confidence_levels:

    for name, model in best_models.items():
        print(f"Model: {name}, Confidence level: {confidence_level}")

        cp_model = SplitConformalRegressor(
            estimator=model,
            confidence_level=confidence_level,
            prefit=True)

        cp_model.conformalize(X_cal, y_cal)

        y_pred, intervals = cp_model.predict_interval(X_test)

        lower = intervals[:, 0, 0]
        upper = intervals[:, 1, 0]

        mae = mean_absolute_error(y_test, y_pred)
        rmse = np.sqrt(mean_squared_error(y_test, y_pred))
        r2 = r2_score(y_test, y_pred)

        coverage = ((y_test >= lower) & (y_test <= upper)).mean()
        avg_width = np.mean(upper - lower)
```

```

coverage_results.append({
    "Model": name,
    "Confidence Level": confidence_level,
    "MAE": mae,
    "RMSE": rmse,
    "R2": r2,
    "Coverage": coverage,
    "Average Interval Width": avg_width})

coverage_results_df = pd.DataFrame(coverage_results)
coverage_results_df

```

Model: Random Forest, Confidence level: 0.8

Model: XGBoost, Confidence level: 0.8

Model: MLP, Confidence level: 0.8

Model: Random Forest, Confidence level: 0.9

Model: XGBoost, Confidence level: 0.9

Model: MLP, Confidence level: 0.9

Model: Random Forest, Confidence level: 0.95

Model: XGBoost, Confidence level: 0.95

Model: MLP, Confidence level: 0.95

Out[]:

	Model	Confidence Level	MAE	RMSE	R2	Coverage	Average Interval Width
0	Random Forest	0.80	2.327295	3.068703	0.660717	0.804312	7.542449
1	XGBoost	0.80	2.313274	3.089012	0.656212	0.797678	7.398717
2	MLP	0.80	2.267956	3.038078	0.667455	0.819237	7.585301
3	Random Forest	0.90	2.327295	3.068703	0.660717	0.885572	9.923735
4	XGBoost	0.90	2.313274	3.089012	0.656212	0.883914	9.681747
5	MLP	0.90	2.267956	3.038078	0.667455	0.888889	9.611099
6	Random Forest	0.95	2.327295	3.068703	0.660717	0.945274	12.507185
7	XGBoost	0.95	2.313274	3.089012	0.656212	0.930348	11.919070
8	MLP	0.95	2.267956	3.038078	0.667455	0.936982	11.797820

```

In [88]: coverage_results_df.sort_values(
          by=["Model", "Confidence Level"])

```

Out[88]:

	Model	Confidence Level	MAE	RMSE	R2	Coverage	Average Interval Width
2	MLP	0.80	2.267956	3.038078	0.667455	0.819237	7.585301
5	MLP	0.90	2.267956	3.038078	0.667455	0.888889	9.611099
8	MLP	0.95	2.267956	3.038078	0.667455	0.936982	11.797820
0	Random Forest	0.80	2.327295	3.068703	0.660717	0.804312	7.542449
3	Random Forest	0.90	2.327295	3.068703	0.660717	0.885572	9.923735
6	Random Forest	0.95	2.327295	3.068703	0.660717	0.945274	12.507185
1	XGBoost	0.80	2.313274	3.089012	0.656212	0.797678	7.398717
4	XGBoost	0.90	2.313274	3.089012	0.656212	0.883914	9.681747
7	XGBoost	0.95	2.313274	3.089012	0.656212	0.930348	11.919070

```

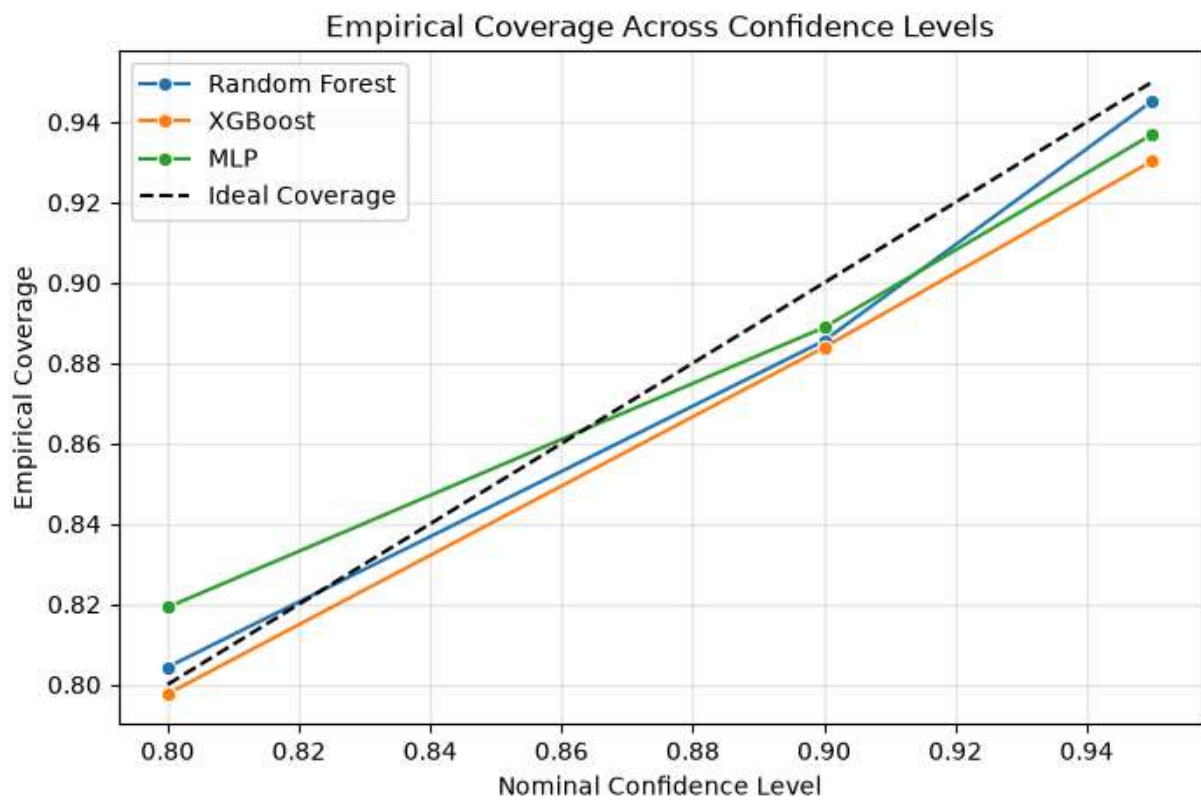
In [ ]: plt.figure(figsize=(8, 5))

sns.lineplot(
    data=coverage_results_df,
    x="Confidence Level",
    y="Coverage",
    hue="Model",
    marker="o")

plt.plot([0.80, 0.95], [0.80, 0.95], linestyle="--", color="black", label="Ideal Co

plt.title("Empirical Coverage Across Confidence Levels")
plt.xlabel("Nominal Confidence Level")
plt.ylabel("Empirical Coverage")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()

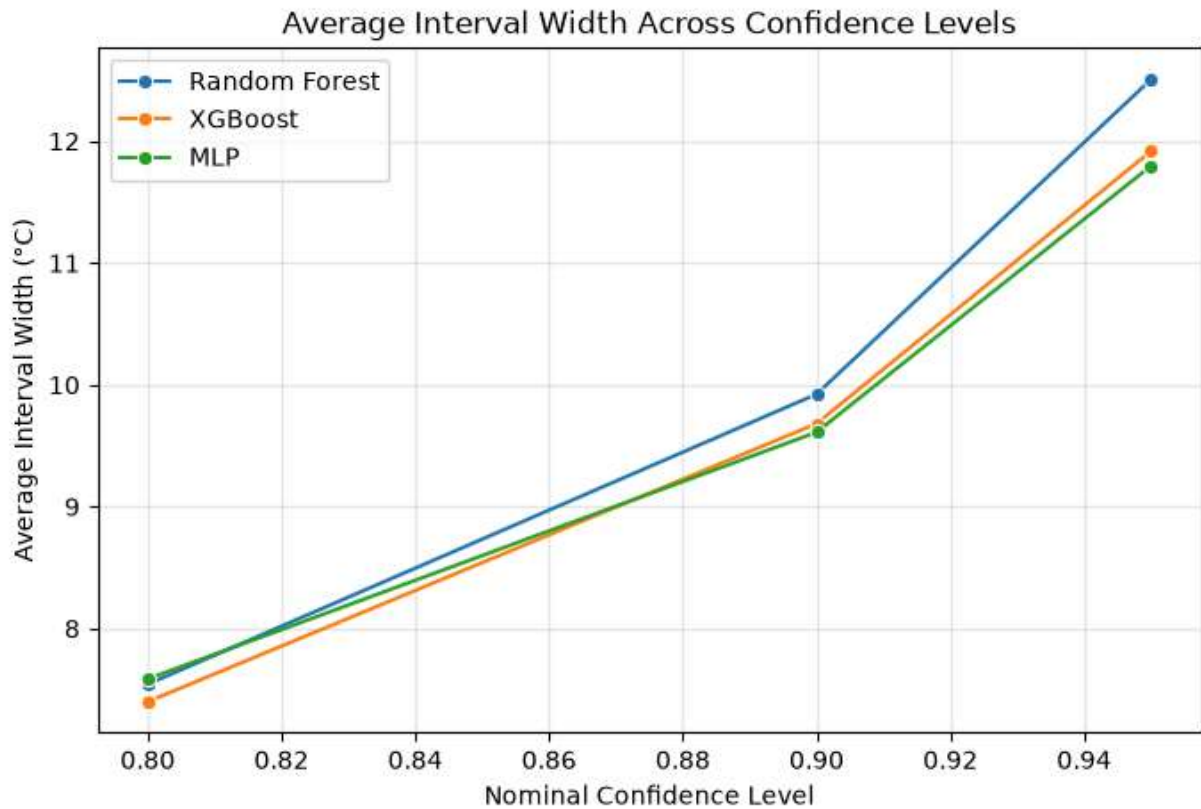
```



```
In [ ]: plt.figure(figsize=(8, 5))

sns.lineplot(
    data=coverage_results_df,
    x="Confidence Level",
    y="Average Interval Width",
    hue="Model",
    marker="o")

plt.title("Average Interval Width Across Confidence Levels")
plt.xlabel("Nominal Confidence Level")
plt.ylabel("Average Interval Width (°C)")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```



Ensemble model

```
In [93]: from sklearn.ensemble import VotingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from mapie.regression import SplitConformalRegressor
import numpy as np
import pandas as pd

ensemble_model = VotingRegressor([
    ("rf", best_models["Random Forest"]),
    ("xgb", best_models["XGBoost"]),
    ("mlp", best_models["MLP"])
])

ensemble_model.fit(X_train, y_train)

ensemble_pred = ensemble_model.predict(X_test)

ensemble_mae = mean_absolute_error(y_test, ensemble_pred)
ensemble_rmse = np.sqrt(mean_squared_error(y_test, ensemble_pred))
ensemble_r2 = r2_score(y_test, ensemble_pred)

print("Ensemble Point Prediction Performance")
print("MAE:", ensemble_mae)
print("RMSE:", ensemble_rmse)
print("R2:", ensemble_r2)
```

Ensemble Point Prediction Performance

MAE: 2.266263898228041

RMSE: 3.02470104370586

R2: 0.6703774474112274

```

In [94]: cp_ensemble = SplitConformalRegressor(
            estimator=ensemble_model,
            confidence_level=0.90,
            prefit=True
        )

cp_ensemble.conformalize(X_cal, y_cal)

ensemble_pred, ensemble_intervals = cp_ensemble.predict_interval(X_test)

ensemble_lower = ensemble_intervals[:, 0, 0]
ensemble_upper = ensemble_intervals[:, 1, 0]

ensemble_coverage = ((y_test >= ensemble_lower) & (y_test <= ensemble_upper)).mean()
ensemble_width = np.mean(ensemble_upper - ensemble_lower)

ensemble_results = {
    "Model": "Ensemble",
    "MAE": mean_absolute_error(y_test, ensemble_pred),
    "RMSE": np.sqrt(mean_squared_error(y_test, ensemble_pred)),
    "R2": r2_score(y_test, ensemble_pred),
    "Coverage": ensemble_coverage,
    "Average Interval Width": ensemble_width
}

ensemble_results

```

```

Out[94]: {'Model': 'Ensemble',
          'MAE': 2.266263898228041,
          'RMSE': np.float64(3.02470104370586),
          'R2': 0.6703774474112274,
          'Coverage': np.float64(0.8905472636815921),
          'Average Interval Width': np.float64(9.794980445129676)}

```

```

In [95]: ensemble_results_df = pd.DataFrame([ensemble_results])

final_results_df = pd.concat(
    [tuned_results_df, ensemble_results_df],
    ignore_index=True)

final_results_df

```

```

Out[95]:

```

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
0	Random Forest	2.327295	3.068703	0.660717	0.885572	9.923735
1	XGBoost	2.313274	3.089012	0.656212	0.883914	9.681747
2	MLP	2.267956	3.038078	0.667455	0.888889	9.611099
3	Ensemble	2.266264	3.024701	0.670377	0.890547	9.794980

```

In [96]: ensemble_confidence_results = []

confidence_levels = [0.80, 0.90, 0.95]

for confidence_level in confidence_levels:

    cp_ensemble = SplitConformalRegressor(
        estimator=ensemble_model,
        confidence_level=confidence_level,
        prefit=True
    )

    cp_ensemble.conformalize(X_cal, y_cal)

    y_pred, intervals = cp_ensemble.predict_interval(X_test)

    lower = intervals[:, 0, 0]
    upper = intervals[:, 1, 0]

    coverage = ((y_test >= lower) & (y_test <= upper)).mean()
    avg_width = np.mean(upper - lower)

    ensemble_confidence_results.append({
        "Model": "Ensemble",
        "Confidence Level": confidence_level,
        "MAE": mean_absolute_error(y_test, y_pred),
        "RMSE": np.sqrt(mean_squared_error(y_test, y_pred)),
        "R2": r2_score(y_test, y_pred),
        "Coverage": coverage,
        "Average Interval Width": avg_width
    })

ensemble_confidence_df = pd.DataFrame(ensemble_confidence_results)
ensemble_confidence_df

```

```

Out[96]:

```

	Model	Confidence Level	MAE	RMSE	R2	Coverage	Average Interval Width
0	Ensemble	0.80	2.266264	3.024701	0.670377	0.809287	7.473055
1	Ensemble	0.90	2.266264	3.024701	0.670377	0.890547	9.794980
2	Ensemble	0.95	2.266264	3.024701	0.670377	0.941957	11.941764

Comparison of results before and after uncertainty

```

In [97]: point_only_results = []

for name, model in best_models.items():
    y_pred = model.predict(X_test)

    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))

```

```

r2 = r2_score(y_test, y_pred)

point_only_results.append({
    "Model": name,
    "MAE": mae,
    "RMSE": rmse,
    "R2": r2
})

point_only_results_df = pd.DataFrame(point_only_results)
point_only_results_df

```

Out[97]:

	Model	MAE	RMSE	R2
0	Random Forest	2.327295	3.068703	0.660717
1	XGBoost	2.313274	3.089012	0.656212
2	MLP	2.267956	3.038078	0.667455

```

In [98]: conformal_results_summary = []

confidence_level = 0.90

for name, model in best_models.items():
    cp_model = SplitConformalRegressor(
        estimator=model,
        confidence_level=confidence_level,
        prefit=True
    )

    cp_model.conformalize(X_cal, y_cal)

    y_pred, intervals = cp_model.predict_interval(X_test)

    lower = intervals[:, 0, 0]
    upper = intervals[:, 1, 0]

    coverage = ((y_test >= lower) & (y_test <= upper)).mean()
    avg_width = np.mean(upper - lower)

    conformal_results_summary.append({
        "Model": name,
        "Coverage": coverage,
        "Average Interval Width": avg_width
    })

conformal_results_summary_df = pd.DataFrame(conformal_results_summary)
conformal_results_summary_df

```

Out[98]:

	Model	Coverage	Average Interval Width
0	Random Forest	0.885572	9.923735
1	XGBoost	0.883914	9.681747
2	MLP	0.888889	9.611099

```
In [99]: comparison_results_df = point_only_results_df.merge(
        conformal_results_summary_df,
        on="Model",
        how="left")

comparison_results_df
```

Out[99]:

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
0	Random Forest	2.327295	3.068703	0.660717	0.885572	9.923735
1	XGBoost	2.313274	3.089012	0.656212	0.883914	9.681747
2	MLP	2.267956	3.038078	0.667455	0.888889	9.611099

In [100...

```
point_predictions_df = pd.DataFrame({
    "date": dates_test.values,
    "actual_TMAX": y_test.values
})

for name, model in best_models.items():
    y_pred = model.predict(X_test)
    point_predictions_df[f"{name}_prediction"] = y_pred

point_predictions_df.to_csv("point_predictions_without_uncertainty.csv", index=False)

point_predictions_df.head()
```

Out[100...

	date	actual_TMAX	Random Forest_prediction	XGBoost_prediction	MLP_prediction
0	2024-05-07	19.2	19.016670	20.103079	20.264890
1	2024-05-08	20.8	20.272727	19.763281	20.604705
2	2024-05-09	18.4	21.647816	21.792980	22.150574
3	2024-05-10	18.6	18.457049	17.566505	17.020484
4	2024-05-11	20.6	19.539480	19.819616	19.208161

In [107...

```
uncertainty_predictions = []

confidence_level = 0.90

for name, model in best_models.items():

    cp_model = SplitConformalRegressor(
        estimator=model,
        confidence_level=confidence_level,
        prefit=True
    )

    cp_model.conformalize(X_cal, y_cal)

    y_pred, intervals = cp_model.predict_interval(X_test)

    lower = intervals[:, 0, 0]
    upper = intervals[:, 1, 0]

    temp_df = pd.DataFrame({
        "date": dates_test.values,
        "model": name,
        "actual_TMAX": y_test.values,
        "prediction": y_pred,
        "lower_interval": lower,
        "upper_interval": upper,
        "interval_width": upper - lower,
        "inside_interval": (y_test.values >= lower) & (y_test.values <= upper),
        "confidence_level": confidence_level
    })

    uncertainty_predictions.append(temp_df)

uncertainty_predictions_df = pd.concat(uncertainty_predictions, ignore_index=True)

uncertainty_predictions_df.to_csv("predictions_with_conformal_uncertainty.csv", index=False)

uncertainty_predictions_df.head()
```

Out[107...

	date	model	actual_TMAX	prediction	lower_interval	upper_interval	interval_width
0	2024-05-07	Random Forest	19.2	19.016670	14.054802	23.978538	9.923735
1	2024-05-08	Random Forest	20.8	20.272727	15.310859	25.234594	9.923735
2	2024-05-09	Random Forest	18.4	21.647816	16.685949	26.609684	9.923735
3	2024-05-10	Random Forest	18.6	18.457049	13.495181	23.418917	9.923735
4	2024-05-11	Random Forest	20.6	19.539480	14.577613	24.501348	9.923735



In [105...

```

confidence_levels = [0.80, 0.90, 0.95]

all_uncertainty_predictions = []

for confidence_level in confidence_levels:

    for name, model in best_models.items():

        cp_model = SplitConformalRegressor(
            estimator=model,
            confidence_level=confidence_level,
            prefit=True
        )

        cp_model.conformalize(X_cal, y_cal)

        y_pred, intervals = cp_model.predict_interval(X_test)

        lower = intervals[:, 0, 0]
        upper = intervals[:, 1, 0]

        temp_df = pd.DataFrame({
            "date": dates_test.values,
            "model": name,
            "confidence_level": confidence_level,
            "actual_TMAX": y_test.values,
            "prediction": y_pred,
            "lower_interval": lower,
            "upper_interval": upper,
            "interval_width": upper - lower,
            "inside_interval": (y_test.values >= lower) & (y_test.values <= upper)
        })

        all_uncertainty_predictions.append(temp_df)

all_uncertainty_predictions_df = pd.concat(all_uncertainty_predictions, ignore_index=True)

all_uncertainty_predictions_df.to_csv("predictions_with_uncertainty_80_90_95.csv",

```

```
print(all_uncertainty_predictions_df.head())
```

	date	model	confidence_level	actual_TMAX	prediction	lower_interv
al	upper_interval	interval_width	inside_interval			
0	2024-05-07	Random Forest	0.8	19.2	19.016670	15.2454
45	22.787895	7.542449	True			
1	2024-05-08	Random Forest	0.8	20.8	20.272727	16.5015
02	24.043952	7.542449	True			
2	2024-05-09	Random Forest	0.8	18.4	21.647816	17.8765
92	25.419041	7.542449	True			
3	2024-05-10	Random Forest	0.8	18.6	18.457049	14.6858
24	22.228274	7.542449	True			
4	2024-05-11	Random Forest	0.8	20.6	19.539480	15.7682
56	23.310705	7.542449	True			